

Document Number: P0290R4
Date: 2023-02-10
Author: [Anthony Williams](#)
Audience: LWG

P0290R4: `apply()` for `synchronized_value<T>`

Introduction

This paper is a followup to [P0290R3](#), based on feedback from LWG at the Issaquah 2023 meeting.

This paper targets the Concurrency TS v2.

The basic idea is that `synchronized_value<T>` stores a value of type `T` and a mutex. The `apply()` function then provides a means of accessing the stored value with the mutex locked, automatically unlocking the mutex afterwards.

The `apply()` function is variadic, so you can operate on a set of `synchronized_value<T>` objects. All the mutexes are locked prior to invoking the supplied callable object, and then they are all released afterwards.

The name is chosen to fit in with `std::apply` for tuples, since the operation is conceptually similar. Rather than expanding a `std::tuple` to supply the arguments to the function, the values wrapped by the `synchronized_values` are extracted to supply the arguments to the function.

This provides an easy way for developers to ensure that all accesses to a given object are done with the relevant mutex locked, whilst also allowing for operations that require locks on multiple objects.

In order to avoid simple mistakes when using the `synchronized_value<T>` objects, there are no public member functions or operations other than construction.

The actual implementation may use an alternative synchronization mechanism instead of a mutex, provided that the synchronization requirements are met.

Feedback From Technical Specification

This paper is targetting the C++ Concurrency TS 2, in order to garner feedback from users and implementers. Specifically, feedback is desired for the following questions:

1. Should the `synchronized_value` template be parameterized on the type of the *Lockable* object (defaulted to `std::mutex`)? This would allow use with third party or user-supplied mutexes, but potentially complicates the interface and would require a specialization for the default case if the implementation wanted to use something other than `std::mutex` for performance reasons.
2. Should `apply` accept cv-qualified `synchronized_value<T>` arguments, as well as non-const arguments? This complicates the interface, but allows you to declare a `const synchronized_value<T>` & in order to guarantee that you can't change the stored value through calls to `apply` that are given that reference.

Examples

1: Simple accesses

Simple accesses can be done with simple lambdas:

```
synchronized_value<std::string> s;  
  
std::string read_value(){  
    return apply([](auto& x){return x;},s);  
}  
  
void set_value(std::string const& new_val){  
    apply([&](auto& x){x=new_val;},s);  
}
```

2: More complex processing

More complex processing can be done with a more complex lambda, or a separate function or callable object:

```
synchronized_value<std::queue<message_type>> queue;

void process_message(){
    std::optional<message_type> local_message;
    apply([&](std::queue<message_type>& q){
        if(!q.empty()){
            local_message.emplace(std::move(q.front()));
            q.pop_front();
        }
    },queue);
    if(local_message)
        do_processing(local_message.value());
}
```

3: Multi-value processing

The variadic nature of `apply()` means that writing code that accesses multiple `synchronized_value<T>` objects is straightforward. It uses the same mechanism as `std::lock()` to ensure that the requisite mutexes are locked without deadlock.

The ubiquitous example of transferring money between accounts can then be simply written as a follows:

```
void transfer_money(
    synchronized_value<account>& from_,
    synchronized_value<account>& to_,
    money_value amount){
    apply([=](auto& from,auto& to){
        from.withdraw(amount);
        to.deposit(amount);
    },from_,to_);
}
```

Proposed wording

Add a new row to Table 1 in [general.feature.test] as follows:

Macro name	Value	Header
<code>__cpp_lib_concurrency_v2_synchronized_value</code>	202302	<code><experimental/synchronized_value></code>

Add a new section as follows.

x Synchronized Values

This section describes a class template to provide locked access to a value in order to facilitate the construction of race-free programs.

Header `<experimental/synchronized_value>` synopsis

```
namespace std::experimental::inline concurrency v2 {
    template<class T>
    class synchronized_value;

    template<class F,class ... ValueTypes>
    invoke_result_t<F, ValueTypes&...> apply(
        F&& f,synchronized_value<ValueTypes>&... values);
}
```

x.1 Class template `synchronized_value`

```
namespace std::experimental::inline concurrency v2 {
    template<class T>
    class synchronized_value
    {
    public:
        synchronized_value(synchronized_value const&) = delete;
        synchronized_value& operator=(synchronized_value const&) = delete;

        template<class ... Args>
        synchronized_value(Args&& ... args);
```

```

private:
    T value; // exposition only
    mutex mut; // exposition only
};

template<class T>
synchronized_value(T)
-> synchronized_value<T>;
}

```

An object of type `synchronized_value<T>` wraps an object of type `T`. The wrapped object can be accessed by passing a callable object or function to `apply`. All such accesses are done with a lock held to ensure that only one thread may be accessing the wrapped object for a given `synchronized_value` at a time.

```

template<class ... Args>
synchronized_value(Args&& ... args);

```

Constraints:

- `(sizeof...(Args) != 1) is true OR (!same_as<synchronized_value,remove_cvref_t<Args>> &&...) is true`
- `is_constructible_v<T,Args...> is true`

Effects:

Direct-non-list-initializes `value` with `std::forward<Args>(args)...`.

Throws:

Any exceptions emitted by the initialization of `value`.
`system_error` if any necessary resources cannot be acquired.

x.2 apply function

```

template<class F,class ... ValueTypes>
invoke_result_t<F, ValueTypes&...> apply(
    F&& f,synchronized_value<ValueTypes>&... values);

```

Constraints:

`sizeof...(values) != 0` is true.

Effects:

Equivalent to:

```

    scoped_lock lock(values.mut...);
    return invoke(std::forward<F>(f),values.value...);

```

[Note: A single instance of `synchronized_value` can not be passed more than once to the same invocation of `apply`.

[Example:

```

    synchronized_value<int> sv;
    void f(int,int);
    apply(f,sv,sv); // undefined behaviour, sv passed more than once to same call

```

—End Example] —End Note]

[Note: The invocation of `f` can not call `apply` directly or indirectly passing any of `values...`. —End Note]

Changes since P0290R3

Following discussion in LWG in Issaquah, the following changes have been made:

1. Updated wording to rely on effects clause rather than specify the return value, throws and synchronization properties explicitly.
2. Added a deduction guide.
3. Changed the template parameters to a single pack with constraints in all cases.
4. Added a constraint to the constructor to prevent it being a move constructor or copy constructor.
5. Corrected the header and namespace for putting in the TS rather than IS.
6. Added a feature test macro.
7. Added feedback questions for the TS.

Changes since P0290R2

Following discussion in LWG in Kona, the following changes have been made:

1. Updated wording to match C++23 draft

Changes since P0290R1

Following discussion in LEWG in Kona, the following changes have been made:

1. Fixed HTML typos;
2. Changed signature of `apply` so it overloads `std::apply` for tuples, and is a better match when a `synchronized_value` is supplied.

Changes since P0290R0

Following discussion in SG1 in Oulu, the following changes have been made:

1. The wording has been changed to allow alternative synchronization mechanisms instead of mutexes;
2. Calling `apply` with the same `synchronized_value` more than once in the same argument list is explicitly disallowed;
3. Recursively calling `apply` with an overlapping set of `synchronized_value` objects is explicitly disallowed; and
4. Constructing a `synchronized_value` may throw `std::system_error`.